# 55. Capstone: AI Inference Host on Linux

## 55. Capstone: AI Inference Host on Linux

**Phase:** Phase 12 — Capstones
**Estimated time:** 4–6 days
**Slug:** `55-capstone-ai-host`

### Learning objectives

- Run an inference-friendly Linux host layout
- systemd + reverse proxy + TLS
- Resource limits and logs
- Load test and observe

### Prerequisites

Complete prior numbered lessons. Prefer a real Linux VM/container for Linux-specific topics. Type every command yourself — muscle memory beats skimming.

### Why this matters

A model API is a long-running service with large memory use, slow startup, and requests that may take seconds. These traits make deployment behavior critical. If a deploy kills the process abruptly, in-flight requests fail and clients see errors. If memory is unbounded, one leak kills everything on the host. If limits and logs are missing, nobody can say why latency spiked. This capstone builds a mock inference host (a plain HTTP server is fine, no GPU needed) that demonstrates the full operational pattern.

Shutdown is the part people forget. systemd stops a service by sending **SIGTERM** (TERM), waiting for `TimeoutStopSec`, and only then sending SIGKILL. Your app must catch TERM, stop accepting new requests, finish current ones, and exit cleanly. KILL cannot be caught, so it is the last resort, never the plan.

## Core concepts

| Layer | Setting | Purpose |
|---|---|---|
| Unit | `User=model`, `NoNewPrivileges=true` | least privilege |
| Limits | `MemoryMax=`, `CPUQuota=`, `LimitNOFILE=` | bound the blast radius |
| Stop | `KillSignal=SIGTERM`, `TimeoutStopSec=30` | graceful drain |
| Edge | reverse proxy with TLS on 443 | termination, timeouts |
| Firewall | allow 22 and 443 only | small surface |
| Observability | journald, metrics endpoint, node exporter | capacity facts |
| Operations | runbook for deploy and roll-back | repeatable changes |

```
  client --443/TLS--> [nginx proxy] --> [model server 127.0.0.1:8000]
                                              |  systemd unit, cgroup limits
  deploy: systemctl stop  -> SIGTERM -> app drains requests -> exit 0
                                    \--(30s)--> SIGKILL (last resort)
  watch: memory vs MemoryMax, nofile, p95 latency, 5xx, restart count
```

## Worked examples

Example 1: a mock model server that handles TERM.

```
#!/usr/bin/env python3
import signal, sys, time
from http.server import ThreadingHTTPServer, BaseHTTPRequestHandler

class H(BaseHTTPRequestHandler):
    def do_GET(self):
        time.sleep(2)   # pretend inference
        self.send_response(200); self.end_headers(); self.wfile.write(b"ok\n")

srv = ThreadingHTTPServer(("127.0.0.1", 8000), H)
def stop(sig, frm):
    print("TERM received, draining", flush=True)
    import threading; threading.Thread(target=srv.shutdown).start()
signal.signal(signal.SIGTERM, stop)
srv.serve_forever()
srv.server_close()
print("clean exit", flush=True)
```

Example 2: the unit file.

```
[Unit]
Description=Mock model API
After=network.target

[Service]
User=model
ExecStart=/opt/model/server.py
Restart=on-failure
KillSignal=SIGTERM
TimeoutStopSec=30
MemoryMax=1G
CPUQuota=200%
LimitNOFILE=65536
NoNewPrivileges=true
ProtectSystem=strict
ProtectHome=true
PrivateTmp=true

[Install]
WantedBy=multi-user.target
```

Example 3: test graceful stop for real.

```
sudo systemctl start model
curl -s localhost:8000/ &
sudo systemctl stop model
journalctl -u model -n 5 --no-pager
```

The log should show the TERM message and a clean exit, and the curl should still complete.

Example 4: confirm limits are live.

```
systemctl show model -p MemoryMax -p CPUQuota -p LimitNOFILE
systemd-cgtop -n 1 | head
```

Example 5: lock the surface and check.

```
sudo ufw default deny incoming
sudo ufw allow 22/tcp && sudo ufw allow 443/tcp && sudo ufw enable
sudo ss -tlnp | grep -E ':(8000|443)'
```

## Mini-lab

1. Create the `model` user and install the mock server.
2. Write the unit; start it and verify with `curl`.
3. Put nginx with a self-signed cert in front; proxy to 8000.
4. Allow only 22 and 443 in the firewall.
5. Run a load test: `for i in $(seq 50); do curl -sk https://localhost/ & done; wait`.

6. Run `sudo systemctl stop model` during the load; verify requests finish and the log shows TERM handling.
7. Send `kill -KILL` to a copy and compare: requests fail, no clean message.
8. Lower `MemoryMax` to 50M and trigger an allocation to watch the OOM event in `journalctl`.
9. Write the deploy and rollback runbook.

## Decision guide / failure modes

| Symptom | Likely cause | First check |
|---|---|---|
| Errors during deploy | no TERM handler or short timeout | `journalctl -u`, `TimeoutStopSec` |
| Process killed, no trace | cgroup OOM | `journalctl -k`, `MemoryMax` |
| Too many open files | low nofile | `/proc/PID/limits` |
| Slow under load | CPU quota, disk, cold model | `systemd-cgtop`, `iostat` |
| Port exposed | firewall policy accept | `ss -tlnp`, `nft list ruleset` |

Failure modes: privileged containers; no memory limits; never testing graceful stop; TERM handler that blocks forever; proxy timeout shorter than inference time; and no restart-count alert. Clean behavior under TERM is a rubric item, so demonstrate it with logs.

## Common mistakes

- Privileged containers
- No memory limits
- No graceful shutdown testing

## Exercises

Work in `exercises/55-capstone-ai-host/`. Read that folder's `README.md` fully before coding.

```
cd exercises/55-capstone-ai-host
chmod +x test.sh exercise.sh
./test.sh
```

### What you will build

Open `exercise.sh` and implement `graceful_stop_signal`. It prints the name of the signal, written in uppercase without the `SIG` prefix, that systemd and `kill` send by default to ask a service to shut down cleanly so it can drain in-flight work. One line only. After it passes, finish the capstone: build the mock model server with a handler for that signal, set limits in the unit, and prove with logs that a stop during load lets requests complete. Compare with `kill -KILL` and note the difference. Record limits shown by `systemctl show` and the firewall ports you allow.

**Study loop**

- **A (Drill):** make `./test.sh` pass
- **B (Apply):** run 2–3 lab commands from this lesson on a real Linux host; paste notable output into your notes
- **C (Challenge):** rewrite the exercise from memory the next day

**Do not open `solutions/55-capstone-ai-host/` until you have an honest failing attempt.**

## Self-check

Try to answer without scrolling up:

1. How bound resources?
2. What happens on SIGTERM?
3. Which ports exposed?

## Further reading

- https://www.freedesktop.org/software/systemd/man/systemd.resource-control.html